School of
Management and Law

MDM
Python Refresh

Adrian Moser
adrian.moser@zhaw.ch, FS2026

Inhalt

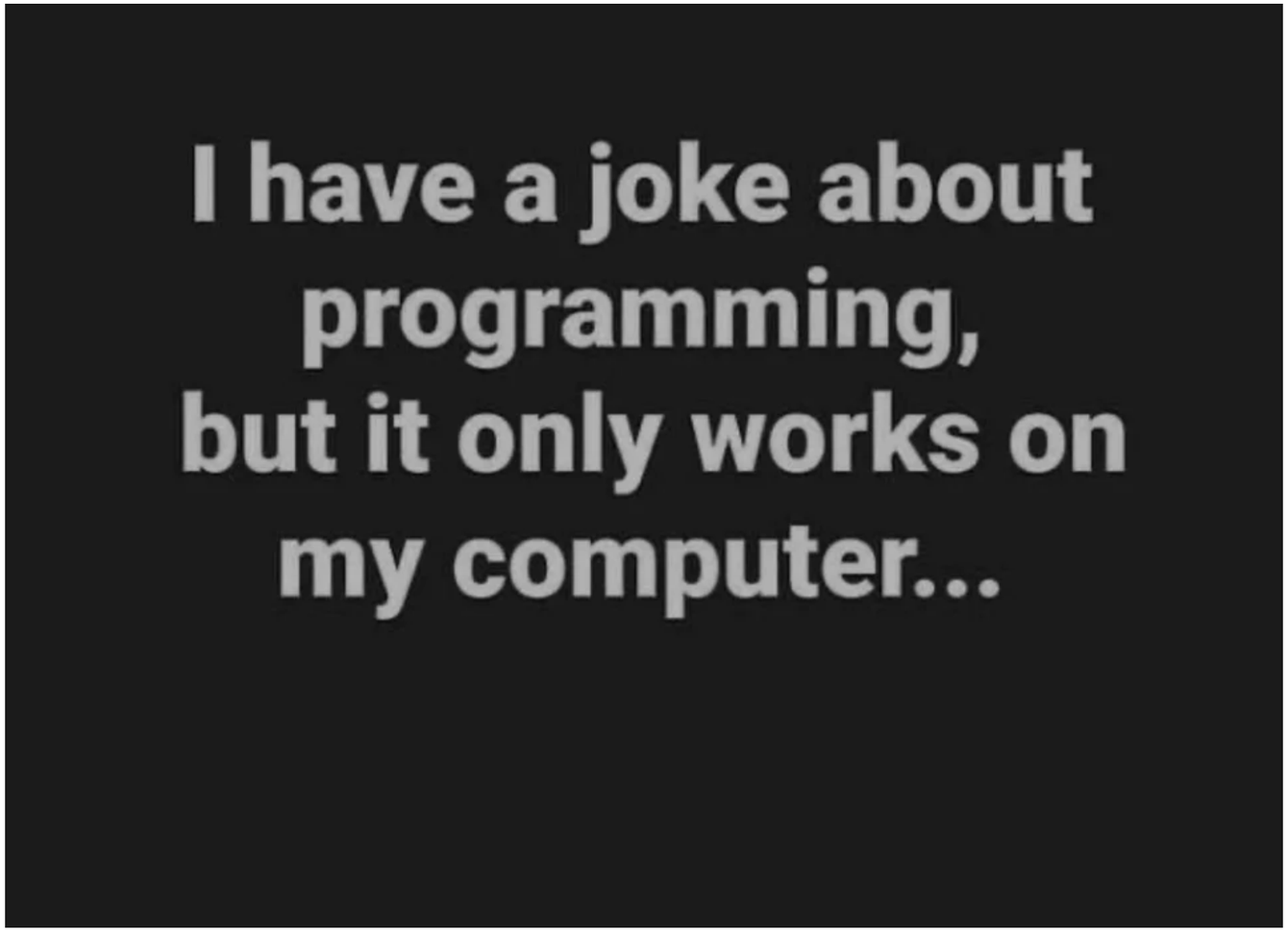
Python Dependency Management

- PyPi, pip, venv, requirements.txt
- VS Code Support
- pip-tools, transitive Dependencies
- Alternativen zu pip
- uv

Python Web Apps mit Flask

- Simple User Interfaces with Bootstrap & Vanilla JS
- Azure Deployment

Warum Dependency Management?



Quelle: Reddit

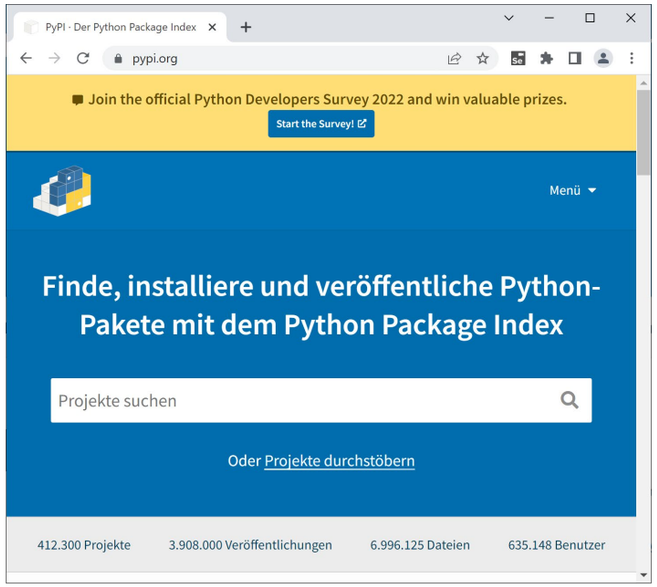
pip install

```
pip install flask
```

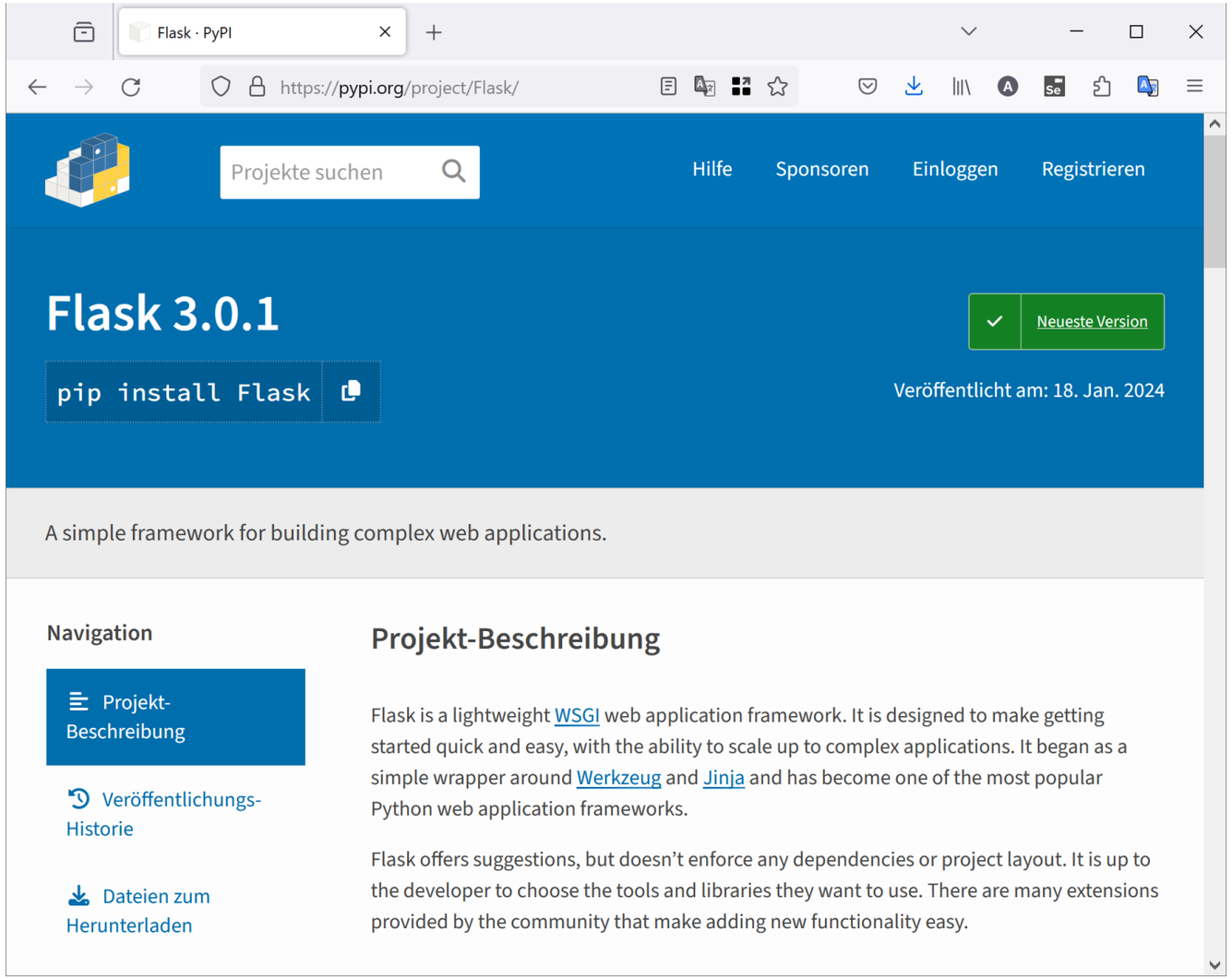
Python Package Index

PyPI

- Python Package Index
- Stellt alle Bibliotheken online zur Verfügung
- Auch bekannt als «Cheese Shop», siehe https://youtu.be/Hz1JWzyvv8A



Versionen


Flask · PyPI
https://pypi.org/project/Flask/
Projekte suchen
Hilfe
Sponsoren
Einloggen
Registrieren
Flask 3.0.1
Neueste Version
pip install Flask
Veröffentlicht am: 18. Jan. 2024
A simple framework for building complex web applications.
Navigation
Projekt-Beschreibung
Veröffentlichungs-Historie
Dateien zum Herunterladen
Projekt-Beschreibung
Flask is a lightweight WSGI web application framework. It is designed to make getting started quick and easy, with the ability to scale up to complex applications. It began as a simple wrapper around Werkzeug and Jinja and has become one of the most popular Python web application frameworks.
Flask offers suggestions, but doesn't enforce any dependencies or project layout. It is up to the developer to choose the tools and libraries they want to use. There are many extensions provided by the community that make adding new functionality easy.

pip show flask

```
PS C:\Users\mosa> pip show flask
Name: Flask
Version: 3.1.2
Summary: A simple framework for building complex web applications.
Home-page:
Author:
Author-email:
License-Expression: BSD-3-Clause
Location: C:\Users\mosa\.pyenv\pyenv-win\versions\3.13.7\Lib\site-packages
Requires: blinker, click, itsdangerous, jinja2, markupsafe, werkzeug
Required-by:
PS C:\Users\mosa>
```

Erste Erkenntnisse

Welche Python-Version verwenden wir für dieses Projekt?

→ pyenv, Python-Version pro Projekt festlegen

Neueste Flask-Version

Standardmässig wird die neueste Flask-Version installiert.

→ Je nach Projektstand eine LTS-Version (Long Term Support) verwenden (sofern verfügbar).

Globale Installation

Flask wird für alle Projekte installiert, die dieselbe Python-Version verwenden.

→ Empfehlung: Virtuelle Umgebungen nutzen, um Abhängigkeiten projektweise zu isolieren.

Transitive Abhängigkeiten, Löschen

→ Flask benötigt zusätzliche Bibliotheken, die automatisch installiert werden

→ pip uninstall flask löscht die transitiven Abhängigkeiten nicht

Lösung: Überprüfung mit pip list, Aufräumen

Übung

```
PS C:\>pip list
Package Version
------- -------
pip     24.3.1
```

```
PS C:\pip list
Package      Version
------------ -------
blinker      1
click        8.
colorama     0.4.
Flask        3.1.
itsdangerous 2
Jinja2       3
MarkupSafe   3.
pip          24.3.1
Werkzeug     3.1.3
```

Bei Verwendung von virtuellen Umgebungen sollte die «Standard-Umgebung» «leer» sein und nur pip selbst enthalten!

Aufräumen:

```
pip freeze > delete.txt
pip uninstall -r delete.txt
```

pyenv und venv

Python Versions-Management
Virtuelle Umgebungen

Lösungen: pyenv und venv (virtuelle Umgebungen)

Übung

Python-Version für Verzeichnis festlegen

```
pyenv versions
pyenv local 3.12.10
python --version
```

Einstellung wird abgelegt in Datei .python-version

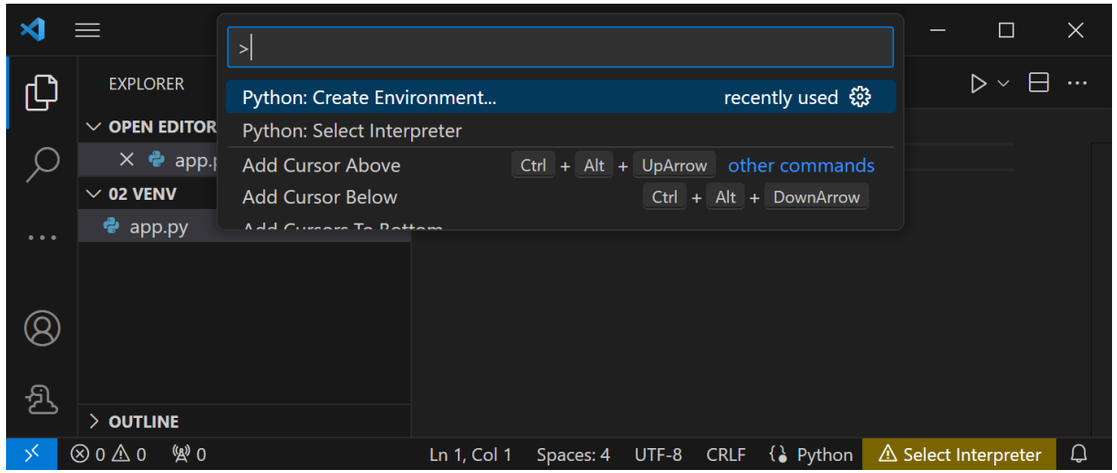
venv

Isolierte Umgebung pro Projekt

Ctrl-Shift-P: **Python: Create Environment…**

Venv

Enter Interpreter Path… (aktuell aufgrund eines Bugs leider notwendig → 3.12.10 auswählen in home/.pyenv)



venv in VS-Code verwenden

Neues Terminal öffnen

```
(.venv) PS C:\...\python venv>
```

… dann flask==3.1.2 installieren

Ist diese Lösung ohne Probleme?

Übung

```
PS C:\>pip list
Package      Version
------------ -------
blinker      1.9.0
click        8.3.1
colorama     0.4.6
Flask        3.1.2
itsdangerous 2.2.0
Jinja2       3.1.6
MarkupSafe   3.0.3
pip          25.3
Werkzeug     3.1.5
```

```
PS C:\> pip show flask
Name: Flask
Version: 3.1.2
Summary: A simple framework for building complex web applications.
Home-page:
Author:
Author-email:
License:
Location: c:\mosa\...\02 flask-playground\.venv\Lib\site-packages
Requires: blinker, click, itsdangerous, Jinja2, Werkzeug
Required-by:
```

Lösen pyenv, venv und pip alle Probleme?

Für einen Entwickler

Einfache Installation von Python-Versionen und PyPI-Packages wie z.B. Flask (PyPI: Python Package Index)

- ✓ **pyenv**: Unterschiedliche Python-Versionen für verschiedene Projekte
- ✓ **venv**: Unterschiedliche Versionen für verschiedene Projekte

Aber…

- Was ist mit den weiteren Entwicklern?
- Wie setzen diese Entwickler das Projekt auf?
- Was ist mit dem Deployment durch den Betrieb?
- Reproduzierbarkeit (bei mehreren Installationen)?
- Sind transitive Dependencies immer identisch?

Erkenntnisse der Lösungen

Für die Entwicklung ausreichend, solange

- Nur **eine** Person entwickelt
- Keine **Installation** auf einem anderen Rechner (z.B. Test, Produktion) gemacht wird

Lösung: **Dependency Management**

- Klare Definition verwendeter Bibliotheken und deren Version
- Reproduzierbarkeit auf anderen Rechnern (Reproducible Install)
- Einfache Installation auf anderen Rechnern

requirements.txt

Eine mögliche Lösung

Erste Lösung: requirements.txt

Übung

pip install

pip install Flask==3.1.2

pip list

Welche Libraries verwendet mein Projekt?

Diese Daten müssen abgelegt und bei einer neuen Installation verwendet werden.

```
PS C:\>pip list
Package      Version
------------ -------
blinker      1.9.0
click        8.3.1
colorama     0.4.6
Flask        3.1.2
itsdangerous 2.2.0
Jinja2       3.1.6
MarkupSafe   3.0.3
pip          25.3
Werkzeug     3.1.5
```

Lösung: requirements.txt

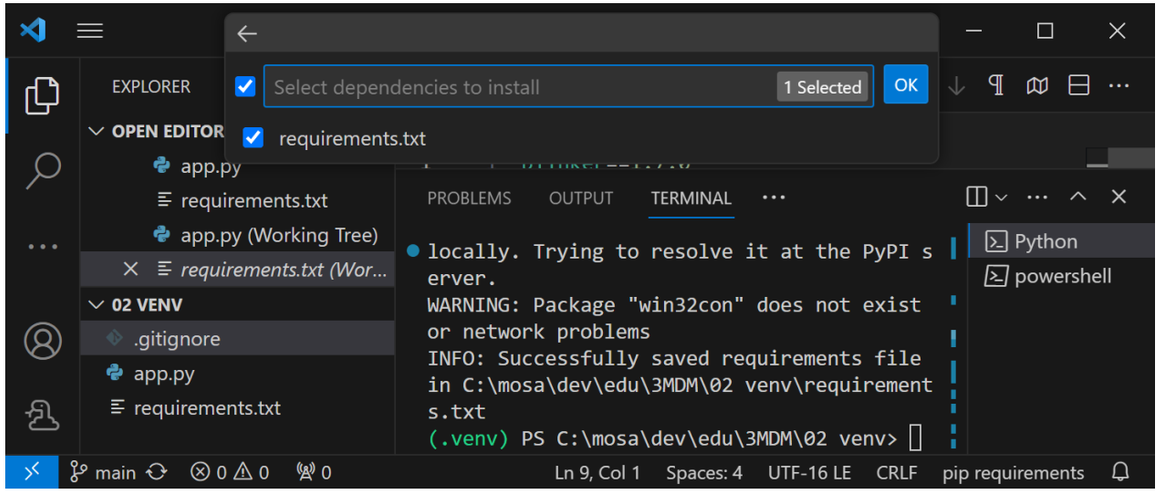
Übung

pip freeze erzeugt eine Datei mit allen Versionen (der Name «requirements.txt» ist eine Konvention)

```
pip freeze > requirements.txt
```

Inhalt **requirements.txt**

```
blinker==1.9.0
click==8.3.1
colorama==0.4.6
Flask==3.1.2
itsdangerous==2.2.0
Jinja2==3.1.6
MarkupSafe==3.0.3
Werkzeug==3.1.5
```



Test: Virtual Environment zurücksetzen

.venv Ordner löschen

Ctrl-Shift-P: Python: Create Environment

requirements.txt anwählen

Überprüfung mit pip list

requirements.txt Optionen

```
# It is possible to specify requirements as plain names.
pytest
pytest-cov
beautifulsoup4

# The syntax supported here is the same as that of requirement specifiers.
docopt == 0.6.1
requests [security] >= 2.8.1, == 2.8.* ; python_version < "2.7"
urllib3 @ https://github.com/urllib3/urllib3/archive/refs/tags/1.26.8.zip

# It is possible to refer to other requirement files or constraints files.
-r other-requirements.txt
-c constraints.txt

# OS-abhängig
PyQt5==5.14.2; platform_system == "Darwin"
PyQt5==5.15.6; platform_system == "Windows"
```

Version weglassen **nicht** empfohlen

Optionale Pakete

URL

Weitere *.txt Dateien

Abhängig vom Betriebssystem

pyenv, venv, requirements.txt – Sind damit alle Probleme gelöst?

pip install / list / freeze

Virtuelle Umgebungen (venv)

requirements.txt

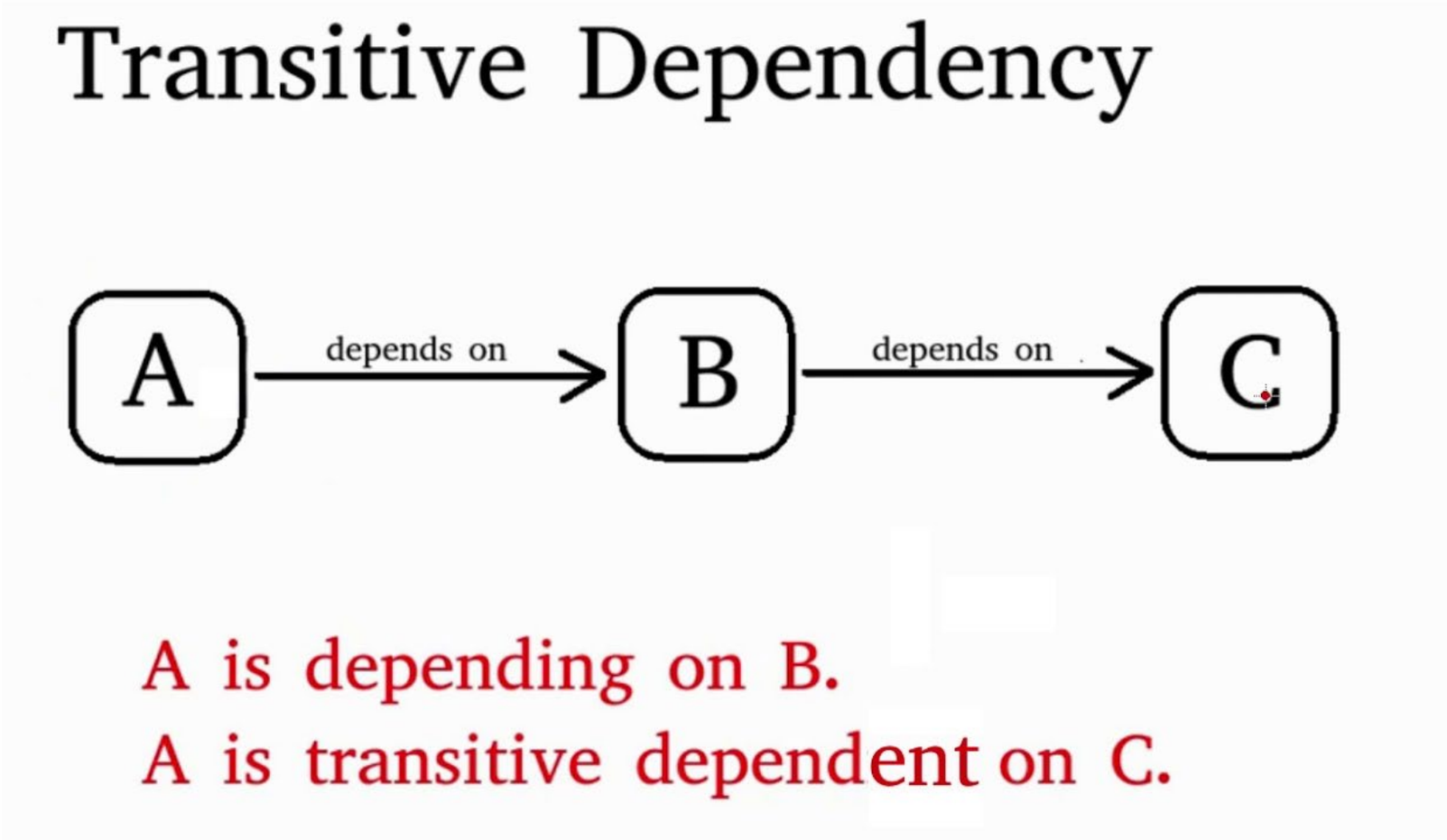
Flask Dependencies

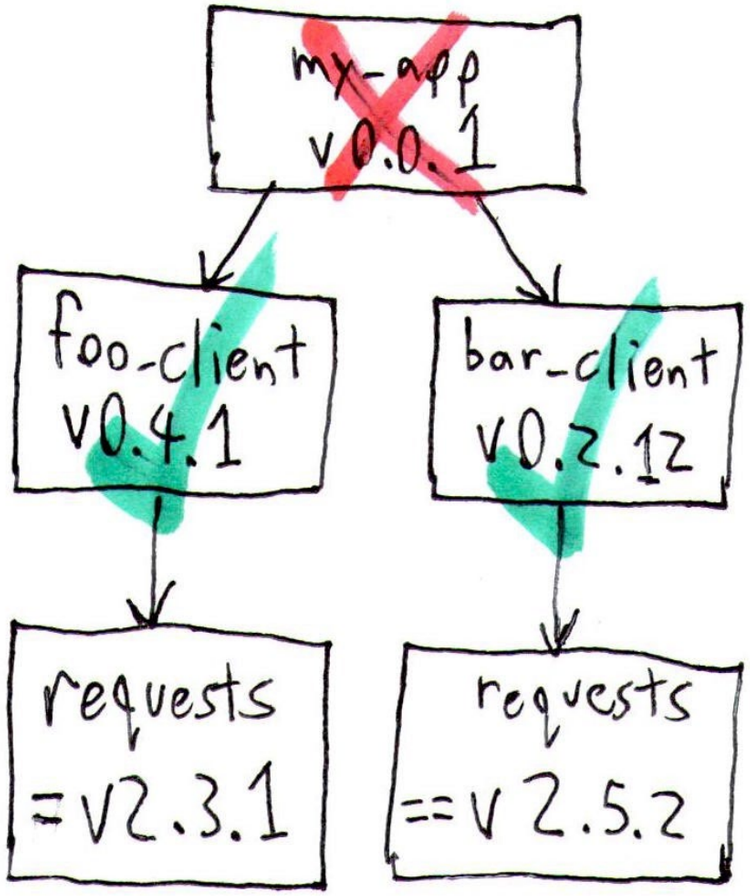
```
flask==3.1.0
flask_cors==5.0.0
scrapy==2.12.0
gpxpy==1.6.2
azure.storage.blob==12.24.0
pandas==2.2.3
seaborn==0.13.2
pymongo==4.10.1
argparse==1.4.0
scikit-learn==1.5.2
```

Transitive Dependencies

```
Package            Version
------------------ -----------
attrs              24.2.0
Automat            24.8.1
azure-core         1.32.0
azure-storage-blob 12.24.0
blinker            1.9.0
certifi            2024.8.30
cffi               1.17.1
charset-normalizer 3.4.0
click              8.1.7
colorama           0.4.6
constantly         23.10.4
contourpy          1.3.1
cryptography       44.0.0
cssselect          1.2.0
cycler             0.12.1
defusedxml         0.7.1
dnspython          2.7.0
filelock           3.16.1
Flask              3.1.0
Flask-Cors         5.0.0
fonttools          4.55.0
gpxpy              1.6.2
hyperlink          21.0.0
idna               3.10
incremental        24.7.2
isodate            0.7.2
itemadapter        0.10.0
itemloaders        1.3.2
itsdangerous       2.2.0
Jinja2             3.1.4
jmespath           1.0.1
joblib             1.4.2
kiwisolver         1.4.7
lxml               5.3.0
MarkupSafe         3.0.2
matplotlib         3.9.3
numpy              2.1.3
packaging          24.2
pandas             2.2.3
parsel             1.9.1
pillow             11.0.0
pip                24.3.1
Protego            0.3.1
pyasn1             0.6.1
pyasn1_modules     0.4.1
pycparser          2.22
PyDispatcher       2.0.7
pymongo            4.10.1
pyOpenSSL          24.3.0
pyparsing          3.2.0
python-dateutil    2.9.0.post0
pytz               2024.2
queuelib           1.7.0
requests           2.32.3
requests-file      2.1.0
scikit-learn       1.5.2
scipy              1.14.1
Scrapy             2.12.0
seaborn            0.13.2
service-identity   24.2.0
setuptools         75.6.0
six                1.16.0
threadpoolctl      3.5.0
tldextract         5.1.3
Twisted            24.11.0
typing_extensions  4.12.2
tzdata             2024.2
urllib3            2.2.3
w3lib              2.2.1
Werkzeug           3.1.3
zope.interface     7.2
```

Transitive Dependency





Lösung von pip-tools: pip-compile

requirements.in → pip-compile requirements.in → generiertes File **requirements.txt**

```
flask==3.1.0
flask_cors==5.0.0
scrapy==2.12.0
gpxpy==1.6.2
azure.storage.blob==12.24.0
pandas==2.2.3
seaborn==0.13.2
pymongo==4.10.1
argparse==1.4.0
scikit-learn==1.5.2
```

Die meisten pip-Optionen, wie z.B. --find-links (zusätzliche URLs, von denen pip Pakete laden kann) werden unterstützt.

```
#
# This file is autogenerated by pip-compile
with Python 3.13
# by the following command:
#
#    pip-compile requirements.in
#
argparse==1.4.0
    # via -r requirements.in
attrs==24.2.0
    # via
    #   service-identity
    #   twisted
automat==24.8.1
...
```

pip-tools als Developer Dependency

Woher kommt pip-tools?

Entweder dennoch global installieren oder in **dev-requirements.in** definieren.

```
pip-tools==7.4.1
```

Alle Abhängigkeiten bereinigen:

```
requirements.txt löschen
.venv erstellen (mit VS Code)
pip install -r dev-requirements.in
pip-compile requirements.in
pip install -r requirements.txt
```

Abhängigkeiten installieren:

```
.venv erstellen (mit VS Code)
pip install -r requirements.txt
```

Python-Projekt mit requirements.txt auf GitHub pushen

pip und Git

- Nur **dev-requirements.in, requirements.in, requirements.txt** gehört ins Git Repository
- Virtualenv in **.gitignore** ausschliessen (Name des Verzeichnisses)
- https://raw.githubusercontent.com/github/gitignore/main/Python.gitignore

```
# Python
__pycache__/
.venv
```

Auf GitHub pushen

Ctrl-Shift-P

Publish to GitHub

Zwischenstand Python Dependency Management

Sind mit

pyenv
venv
pip-tools
requirements.in
requirements.txt

alle Probleme gelöst?

Reproduzierbare Installationen

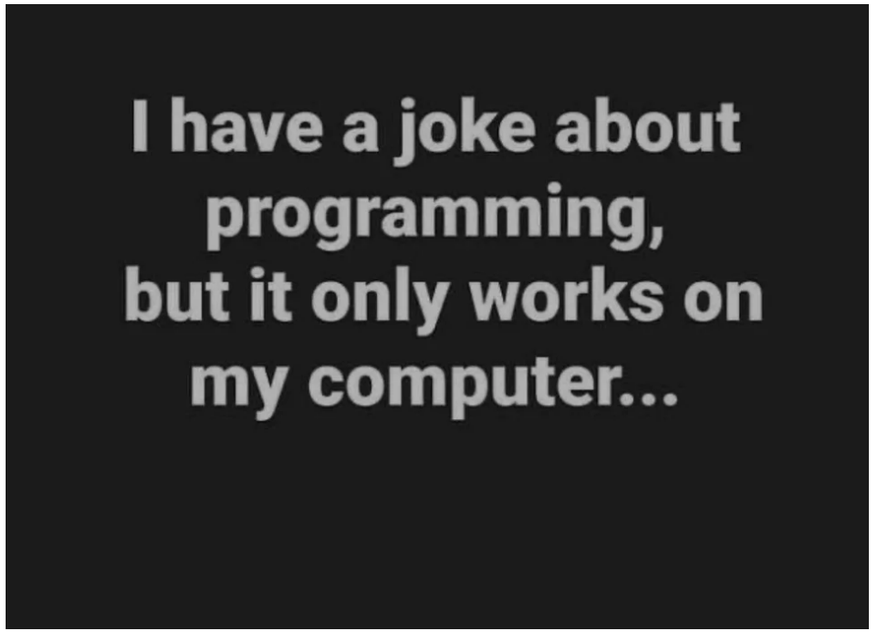
Verbleibendes Problem – Abhängigkeiten können nicht fixiert sein (==)

Dies ist insbesondere bei transitiven Abhängigkeiten ein Problem (nicht unter meiner Kontrolle)

```
requests = ">=2.28,<3.0"
```

→ *Works on my machine*“-Probleme

→ pip vertraut darauf, dass PyPI schon „das Richtige“ liefert



Lösungsidee für das Problem

Lock Files

Auswege aus der Python Dependency Hell:
Vertrauen ist GUT, Kontrolle ist BESSER!

Lock-Files

Idee

Exakte Paketversionen	→ (requests==2.31.0)
Transitive Abhängigkeiten	→ (urllib3, certifi, idna)
Kryptografische Hashes	→ (sha256:… verifiziert Paketinhalt, entspricht Fingerabdruck)
Plattform-Marker	→ (python>=3.10, sys_platform=="linux")
Quellen	→ (PyPI, privater Index, Git-Commit)

Reproduzierbarer Zustand → **Alle Bibliotheken sind identisch in Dev / CI / Prod**

Beispiel für Lock-File

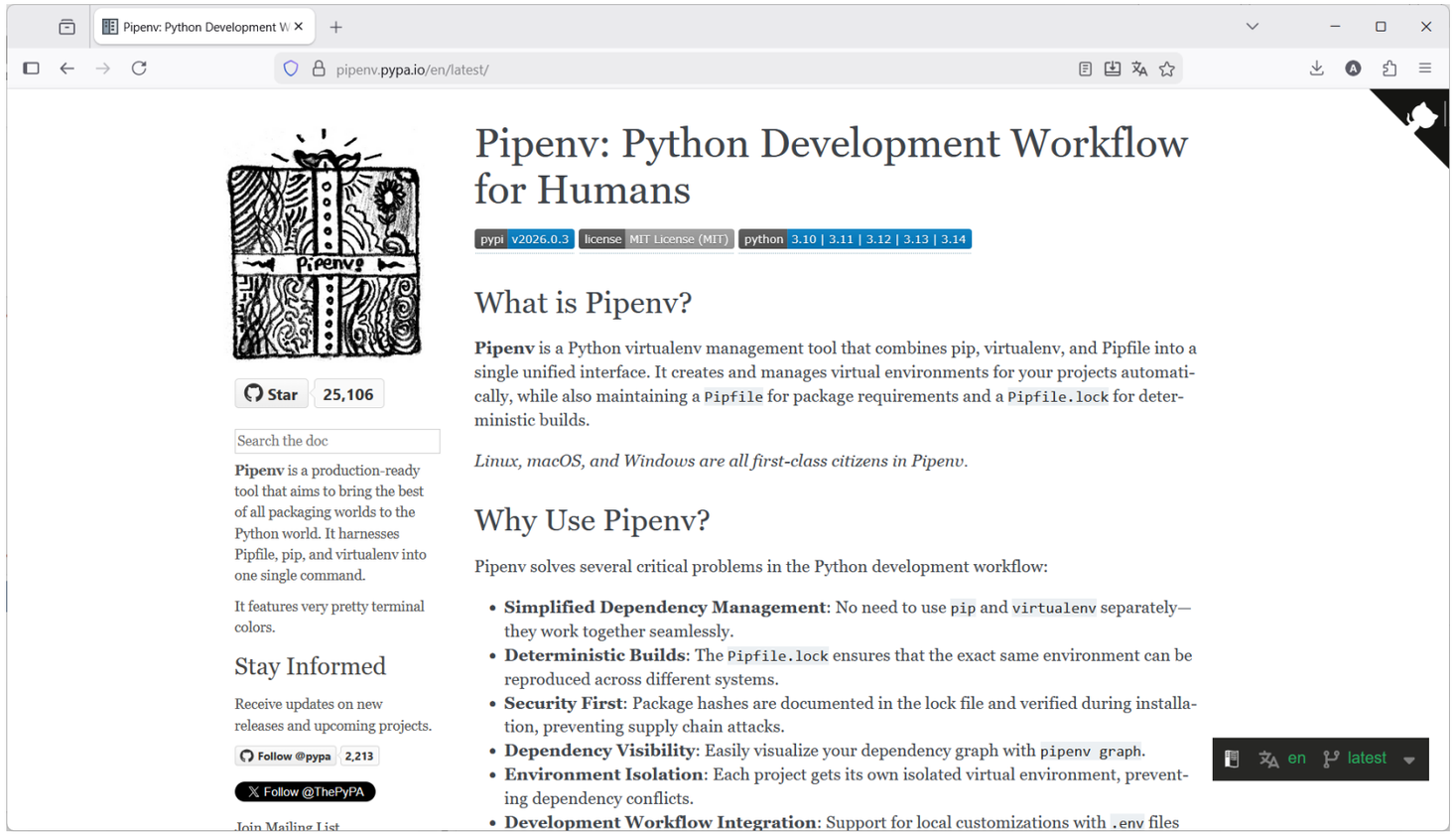
```
[[package]]
name = "requests"
version = "2.31.0"
source = "registry+https://pypi.org/simple"

sdist = {
  url = "https://files.pythonhosted.org/packages/.../requests-2.31.0.tar.gz",
  hash = "sha256:58cd2187c01e70e6e1b6f5b6a8c1e9e3c9d5c5a0c9f6e8f3a1b2c4d5e6f7a"
}

wheels = [
  {
    url = "https://files.pythonhosted.org/packages/.../requests-2.31.0-py3-none-any.whl",
    hash = "sha256:942c5a6d3b8e4c1f0a9e7d6b5a4c3f2e1d0c9b8a7f6e5d4c3b2a1f0e9d8c7b6"
  }
]
```

Lock: Lösungsansätze verschiedener Tools

Tool	Lockfile	Hashes	Transitive Abhängigkeiten versioniert
pip	requirements.txt	optional	nein
pip-tools	requirements.txt*	optional (empfohlen)*	ja
pipenv	Pipfile.lock	ja	ja
poetry	poetry.lock	ja	ja
conda	conda-lock.yml**	nein	ja
uv	uv.lock	ja	ja

* mit pip-compile --generate-hashes
** via conda-lock

Pipenv


Pipenv: Python Development W
pipenv.pypa.io/en/latest/
Pipenv: Python Development Workflow for Humans
pypi v2026.0.3
license MIT License (MIT)
python 3.10 | 3.11 | 3.12 | 3.13 | 3.14
What is Pipenv?
Pipenv is a Python virtualenv management tool that combines pip, virtualenv, and Pipfile into a single unified interface. It creates and manages virtual environments for your projects automatically, while also maintaining a Pipfile for package requirements and a Pipfile.lock for deterministic builds.
Linux, macOS, and Windows are all first-class citizens in Pipenv.
Why Use Pipenv?
Pipenv solves several critical problems in the Python development workflow:
Simplified Dependency Management: No need to use pip and virtualenv separately—they work together seamlessly.
Deterministic Builds: The Pipfile.lock ensures that the exact same environment can be reproduced across different systems.
Security First: Package hashes are documented in the lock file and verified during installation, preventing supply chain attacks.
Dependency Visibility: Easily visualize your dependency graph with pipenv graph.
Environment Isolation: Each project gets its own isolated virtual environment, preventing dependency conflicts.
Development Workflow Integration: Support for local customizations with .env files
Star 25,106
Search the doc
Pipenv is a production-ready tool that aims to bring the best of all packaging worlds to the Python world. It harnesses Pipfile, pip, and virtualenv into one single command.
It features very pretty terminal colors.
Stay Informed
Receive updates on new releases and upcoming projects.
Follow @pypa 2,213
Follow @ThePyPA
Join Mailing List
en
latest

pipenv

Pipfile

- Enthält Dependencies und weitere Informationen

Pipfile.lock

- Enthält exakte Version inklusive Hash-Wert (Reproduzierbarkeit)

Wichtige Befehle

- pip install pipenv
- pipenv shell
- pipenv install «package»

```
[[source]]
url =
https://pypi.org/simple
verify_ssl = true
name = "pypi«

[packages]
flask = "*"
gunicorn = "*"
pandas = "*"

[dev-packages]

[requires]
python_version = "3.8"
```

pipenv Pipfile.lock

```
"flask": {
        "hashes": [
            "sha256:7b2fb8e934ddd50731893bdcdb00fc8c0315916f9fcd50d22c7cc1a95ab634e2",
            "sha256:cb90f62f1d8e4dc4621f52106613488b5ba826b2e1e10a33eac92f723093ab6a"
        ],
        "index": "pypi",
        "version": "==2.0.2"
    },
```

SHA256 ist eine **Hashfunktion**.

Eine Abbildung, die eine grosse Eingabemenge auf eine kleinere Zielmenge (Hashwert) abbildet.

Hashfunktionen werden in der Informationstechnik vorrangig dazu benutzt, die Integrität von Daten zu garantieren.

pipenv Beispiel

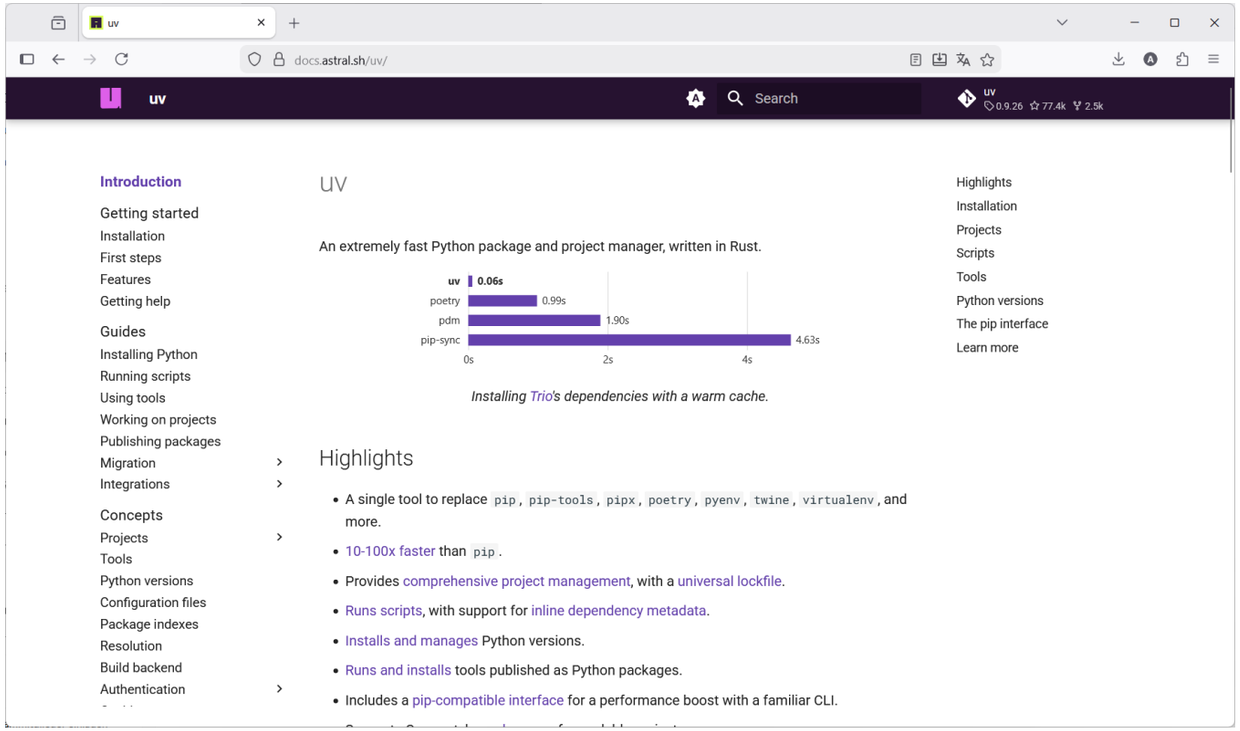
Pipfile.lock

```
{
    "_meta": {
        "hash": {
            "sha256": "44663e029cea5644fccc83b1d4ba7c71e5fb50d120
        },
        "pipfile-spec": 6,
        "requires": {
            "python_version": "3.8"
        },
        "sources": [
            {
                "name": "pypi",
                "url": "https://pypi.org/simple",
                "verify_ssl": true
            }
        ]
    },
    "default": {
        "click": {
            "hashes": [
                "sha256:353f466495adaeb40b6b5f592f9f91cb22372351c84caeb068132442a4518ef3",
                "sha256:410e932b050f5eed773c4cda94de75971c89cdb3155a72a0831139a79e5ecb5b"
            ],
            "markers": "python_version >= '3.6’”,
...
```

```
[[source]]
url = "https://pypi.org/simple"
verify_ssl = true
name = "pypi"

[packages]
flask = "*"
gunicorn = "*"
pandas = "*"

[dev-packages]

[requires]
python_version = "3.8"
```

uv

“An extremely fast Python package and project manager, written in Rust”



uv Hello World mit Flask

Übung

1. Neuen Ordner erstellen
2. in VS Code öffnen
3. MDM Profile Python wählen

4. uv Projekt erstellen

```
# Python-Version setzen (pyenv)
pyenv local 3.13.7

# uv Projekt anlegen
uv init

# Flask
uv add flask==3.1.2
```

5. main.py ersetzen:

```
from flask import Flask
app = Flask(__name__)
@app.route("/")
def hello_world():
    return "<p>Hello, World!</p>"
```

6. launch.json erstellen

uv Analyse

pyproject.toml ersetzt requirements.txt / Pipfile

```
[project]
name = "02-uv-helloworld"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.13.7"
dependencies = [
    "flask==3.1.2",
]
```

→ uv mit pyproject.toml

→ Alternativ kann uv pip / pip-tools für bestehende Projekte ersetzen

uv.lock (gehört in Git) sorgt für reproduzierbare env

```
[package.metadata]
requires-dist = [{ name = "flask", specifier = "==3.1.2" }]


[[package]]
name = "blinker"
version = "1.9.0"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/21/28/9b3f50ce0e048515135495f198351908d99540d69bfdc8c1d15b73dc55ce/blinker-1.9.0.tar.gz", hash = "sha256:b4ce2265a7abece45e7cc896e98dbebe6cead56bcf805a3d23136d145f5445bf", size = 22460, upload-time = "2024-11-08T17:25:47.436Z" }
wheels = [
    { url = "https://files.pythonhosted.org/packages/10/cb/f2ad4230dc2eb1a74edf38f1a38b9b52277f75bef262d8908e60d957e13c/blinker-1.9.0-py3-none-any.whl", hash = "sha256:ba0efaa9080b619ff2f3459d1d500c57bddea4a6b424b60a91141db6fd2f08bc", size = 8458, upload-time = "2024-11-08T17:25:46.184Z" },
]
```

Flask

Web apps mit Python

Flask Microservice Framework

Flask ist ein populäres Microservice Framework für Python
Vergleichbar mit **Spring Boot Web** für Java und **Express** für JavaScript.

Template Rendering
Im Gegensatz zu einem reinen Microservice Framework kann Flask auch HTML-Seiten generieren mit der Jinja2 Template Engine.

Eine Alternative ist aber die reine Verwendung als REST-Framework und die Darstellung des Frontend mit JavaScript/HTML.

Flask

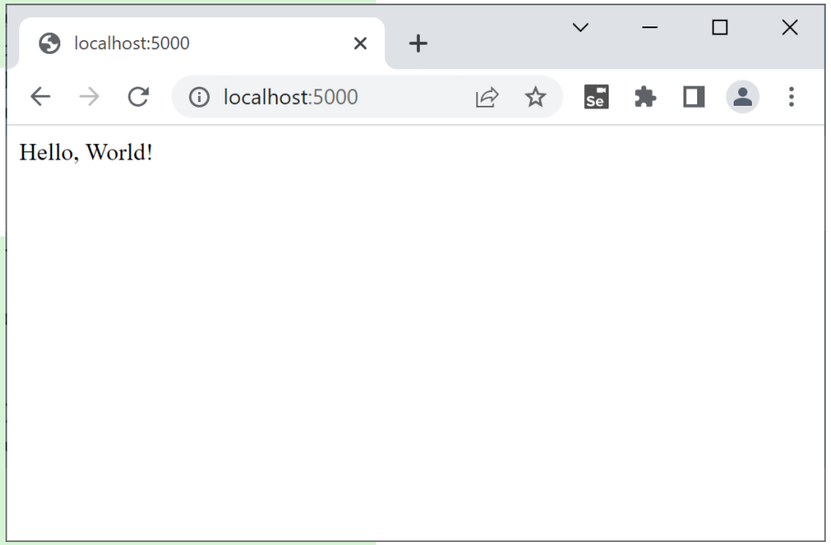
Übung

Projekt erstellen (in neuem Verzeichnis, oder vorheriges Projekt verwenden)

```
pyenv local 3.13.7
uv init
uv add flask==3.1.2
```

app.py

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def hello_world():
    return "<p>Hello, World!</p>"
```



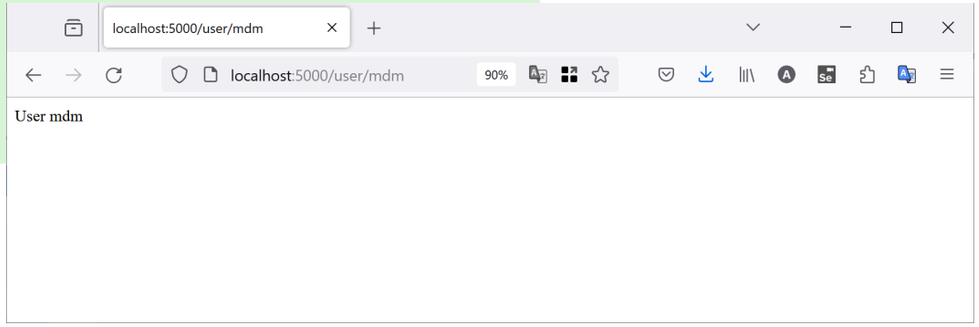
New Terminal, starten mit **flask run** und abrufbar auf http://localhost:5000/ (Ctrl-Klick)

Flask Parameters (Dynamic URLs)

Übung

Die URL kann Parameter (in Flask «Variable Rules» genannt) enthalten

```
@app.route('/user/<username>')
def show_user_profile(username):
    return f'User {username}'

@app.route('/post/<int:post_id>')
def show_post(post_id):
    return f'Post {post_id}'

@app.route('/path/<path:subpath>')
def show_subpath(subpath):
    return f'Subpath {subpath}'
```



Verfügbare Konverter: string (default), int, float, path, uuid

HTTP POST, PUT, GET

Verschiedene HTTP Methoden in der gleichen Methode

```
from flask import request

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        return do_the_login()
    else:
        return show_the_login_form()
```

Unterschiedliche Methoden für unterschiedliche Requests

```
@app.get('/login')
def login_get():
    return show_the_login_form()

@app.post('/login')
def login_post():
    return do_the_login()
```

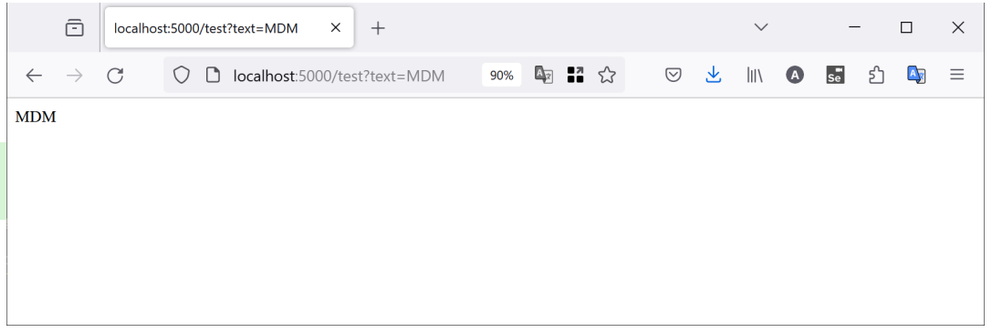
GET Parameter (mit ?)

GET Service

```
@app.route("/test", methods=["GET"])
def test():
    text = request.args.get('text')
    if (text == ""):
        return "Please use text parameter in GET URL"
    return text
```

Aufruf mit

```
http://localhost:5000/test?text=MDM
```



Frontend

HTML

Bootstrap

Single Page Application: index.html

index.html beim Aufruf der App darstellen

```
from flask import Flask
from flask.helpers import send_file

app = Flask(__name__)

@app.route("/")
def indexPage():
    return send_file("frontend/index.html")
```

Bootstrap

Einbindung Bootstrap

```
<!DOCTYPE html>
<html>

<head>
    <title>ML EfficientNet-Lite4</title>
    <meta charset="utf-8" />
    <link rel="stylesheet" href="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/css/bootstrap.min.css"
        integrity="sha384-ggOyR0iXCbMQv3Xipma34MD+dH/1fQ784/j6cY/iJTQUOhcWr7x9JvoRxT2MZw1T"
crossorigin="anonymous">
    <script src="script.js" async></script>
</head>
```

Bootstrap

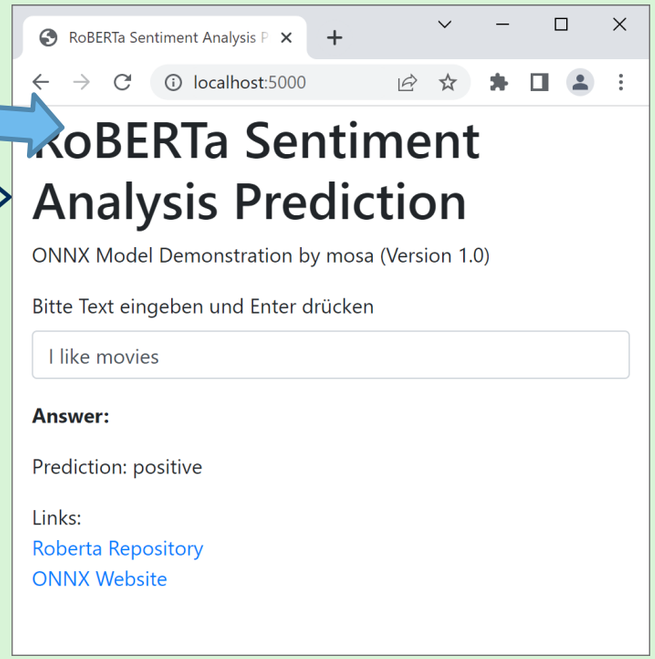
CSS-Framework

Bekannt aus Modul «Prototyping»

Beispiel index.html

Vergleich HTML und UI, JavaScript-Aufruf «onchange»

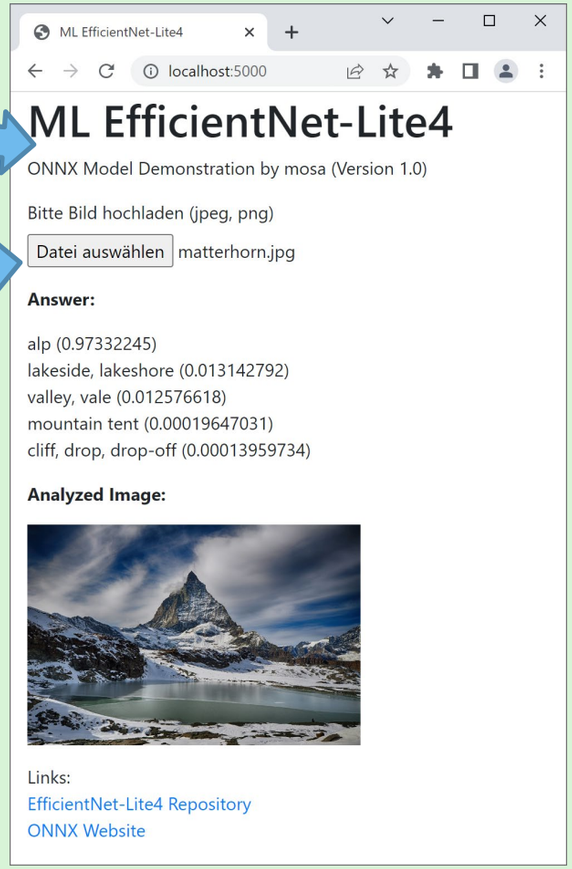
```
<body>
    <div class="container-fluid">
        <h1>RoBERTa Sentiment Analysis Prediction</h1>
        <p>ONNX Model Demonstration by mosa (Version 1.0)</p>

        <div class="form-group">
            <label for="textInput">Bitte Text eingeben und Enter drücken</label>
            <input type="text" class="form-control" id="textInput“
                   onkeyup="getSentiment(event,this.value);">
        </div>

        <div id="answerPart" style="visibility: hidden;">
            <p><b>Answer:</b>
            <div id="answer"></div>
            </p>
        </div>
</div>
</body>
</html>
```



Beispiel index.html

Vergleich HTML und UI, JavaScript-Aufruf «onchange»

```html
<body>
    <div class="container-fluid">
        <h1>ML EfficientNet-Lite4</h1>
        <p>ONNX Model Demonstration by mosa (Version 1.0)</p>

        <form>
            <div class="form-group">
                <label for="exampleFormControlFile1">Bitte Bild hochladen (jpeg, png)</label>
                <input type="file" accept="image/png, image/jpeg"
                         class="form-control-file" id="image" name="image"
                    onchange="checkFiles(this.files);">
            </div>
        </form>

        <div id="answerPart" style="visibility: hidden;">
            <p><b>Answer:</b>
            <div id="answer"></div>
            </p>

            <p id="previewPart"><b>Analyzed Image:</b>
            <p><img id="preview" width="300" src="#" alt="preview" /></p>
            </p>
        </div>
    </div>
</body>
```



JavaScript / Fetch API

Daten von Microservice anfordern und darstellen

JSON

Einbindung JavaScript

<script>-Tag in HTML

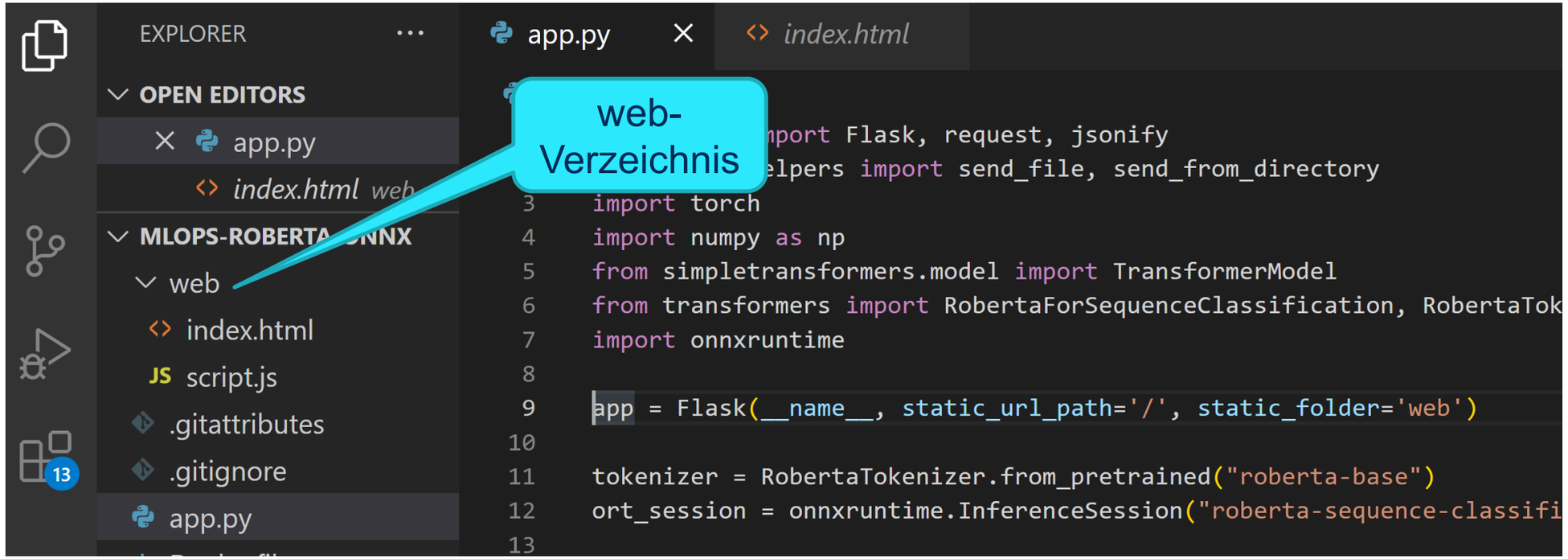
```
<!DOCTYPE html>
<html>

<head>
    <title>ML EfficientNet-Lite4</title>
    <meta charset="utf-8" />
    <link rel="stylesheet" href=“...“ integrity=“..." crossorigin="anonymous">
    <script src="script.js" async></script>
</head>
```

JavaScript in Flask bereitstellen

Folder **web** für statische Dateien konfigurieren

```
app = Flask(__name__, static_url_path='/', static_folder='web')
```

und **script.js** im Ordner **web** platzieren



JavaScript: Text mit GET

```
function getSentiment(event, text) {
    if (!text || event.key !== "Enter") {
        answer.innerHTML = '';
        return;
    }
    answerPart.style.visibility = "visible";

    // Get Sentiment
    fetch('/sentiment?' + new URLSearchParams({
        text: text,
    }), {
        method: 'GET',
        headers: {}
    }).then(
        response => {
            console.log(response)
            response.text().then(function (text) {
                answer.innerHTML = text;
            });

        }
    ).then(
        success => console.log(success)
    ).catch(
        error => console.log(error)
    );
}
```

Sichtbarkeit wechseln

Fetch API

Text zuweisen

Fetch API

- Zugriff auf beliebige Ressourcen, z.B. Netzwerk (GET, POST, …)
- Ersetzt XMLHttpRequest und Framework-Befehle wie jQuery.ajax()

Beispiel

```
fetch('http://example.com/movies.txt')
  .then((response) => {
            console.log(response)
            response.text().then(function (text) {
                answer.innerHTML = text;
            });
  };
```

Dokumentation

https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

JavaScript: Bild mit POST

```
    // Preview
    if (file) {
        preview.src = URL.createObjectURL(files[0])
    }

    // Upload
    const formData = new FormData();
    for (const name in files) {
        formData.append(name, files[name]);
    }

    fetch('/analyze', {
        method: 'POST',
        headers: {
        },
        body: formData
    }).then(
        response => {
            console.log(response)
            response.text().then(function (text) {
                answer.innerHTML = text;
            });

        }
    ).then(
        success => console.log(success)
    ).catch(
        error => console.log(error)
    );
}
```

Bild lokal anzeigen

Fetch API POST

Text zuweisen

Fazit

Für einfache Anwendungen kann ein UI mit folgenden Tools gebaut werden

- Bootstrap
- JavaScript
- Fetch API von JavaScript

Dies wird **Vanilla JS** genannt (http://vanilla-js.com/)

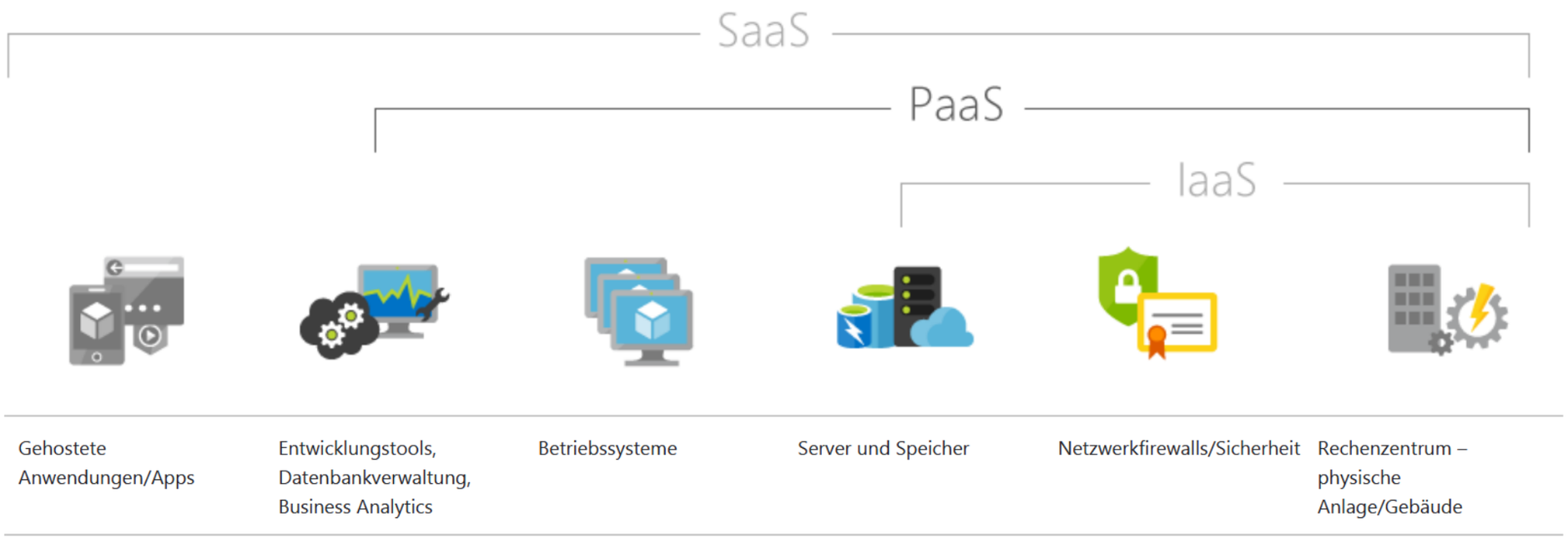
Selbstverständlich dürfen für die Übungen auch Svelte oder andere Frameworks verwendet werden.

Azure App Service

Azure App Service
Begriffe
Python Support
Python Hello World

Azure App Service

- Cloud-Computing-basierte Plattform von Microsoft
- Plattform als Service (PaaS)
- .NET, node.js, PHP, Python, Java und andere



Azure Begriffe

Portal	Web-Applikation zur Konfiguration von Azure Services
Ressource	Teil eines Azure Service, z.B. Datenbank oder VM
Ressourcegruppen	Ein Set von Ressourcen für eine App
Region	Ein oder mehrere Datacenter innerhalb eines Gebietes
Azure CLI	Azure Command Line Interface

Azure und Python

Verfügbare Python Runtimes

```
az webapp list-runtimes --os linux
```

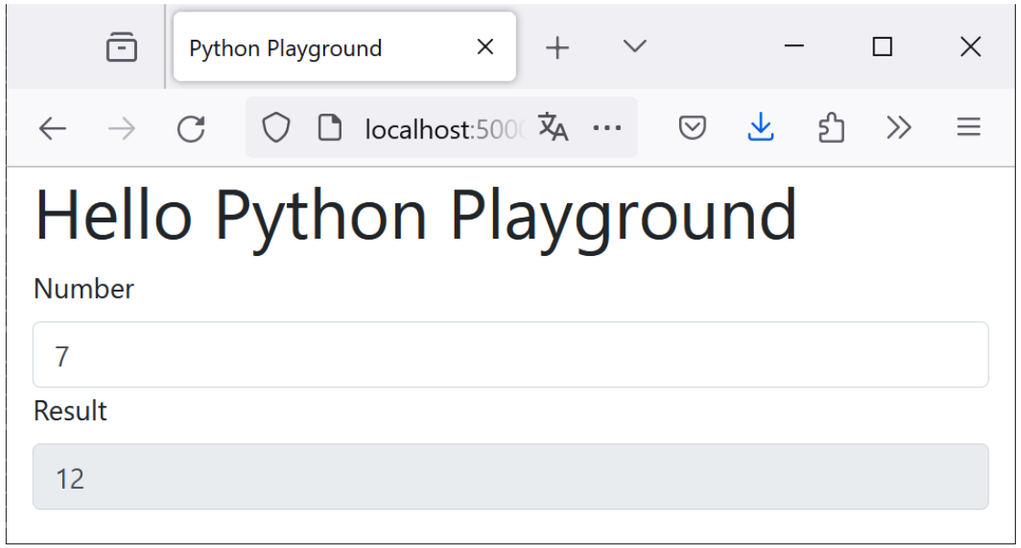
```
[
  …
  "PYTHON:3.12",
  "PYTHON:3.11",
  "PYTHON:3.10",
  "PYTHON:3.9",
  "PYTHON:3.8",
  …
]
```

Azure «Flask Playground»

Übung

https://github.com/mosazhaw/flask-playground

Zählt alle gerade Zahlen der Eingabe zusammen



Azure «Hello World»

Übung

Klonen und starten und localhost:5000

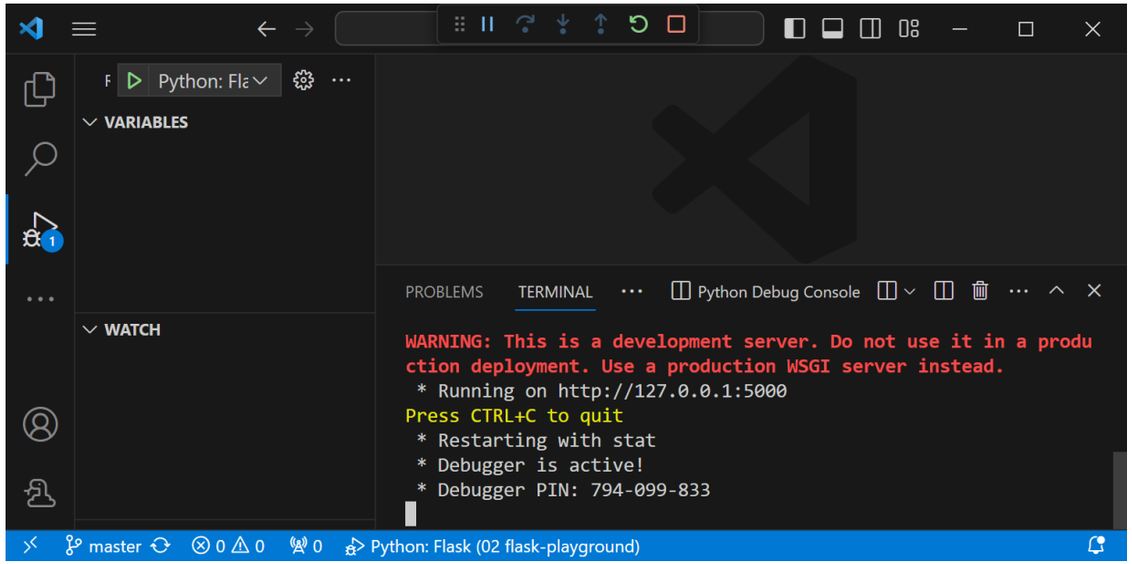
```
git clone https://github.com/mosazhaw/flask-playground
```

Öffnen in VS Code, venv erstellen

App Starten

```
flask run
```

… oder VS Code Launch Config



VS Code: Launch File & Debug

Übung

Run & Debug Sicht

Create a launch json file

Python Debugger wählen

Flask wählen

app.py wählen

Das Launch-json kann auf GitHub abgelegt werden.

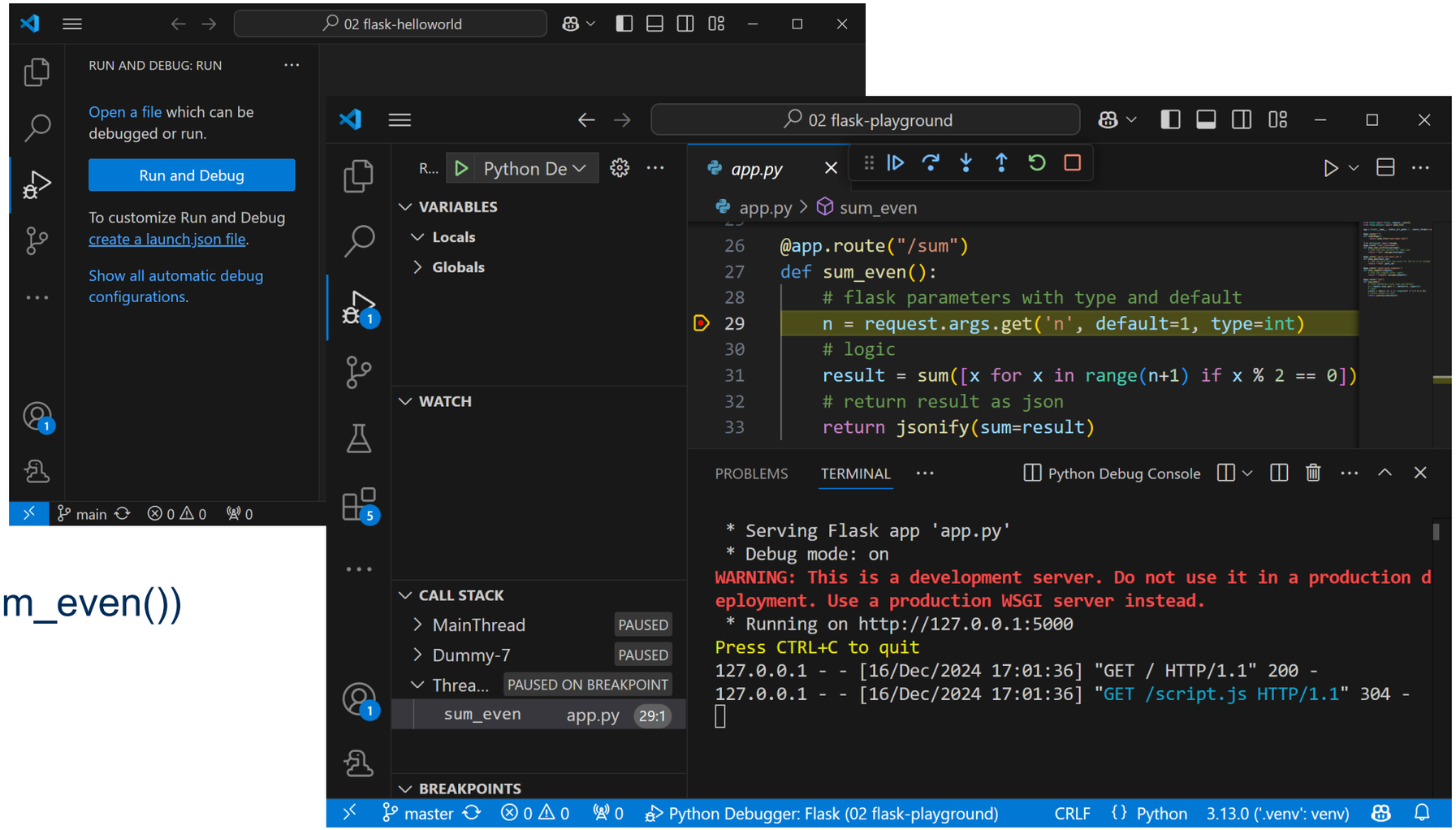
Debug

Breakpoint setzen (z.B. def sum_even())

App starten

Service in Browser aufrufen

App wartet bei Breakpoint



Azure «Hello World»

Übung

Azure Deployment

Eigene Namen wählen

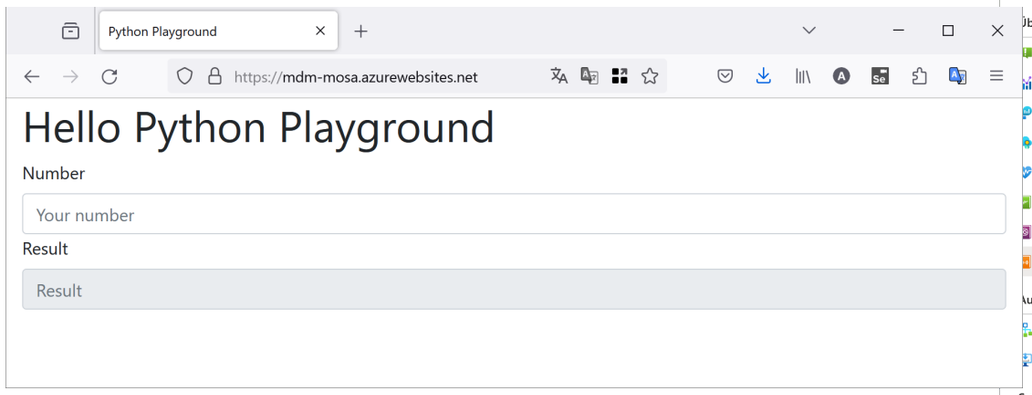
```
az login
az group create --name mdm --location switzerlandnorth
az webapp up --name mdm-mosa --location switzerlandnorth --plan mdm --resource-group mdm --runtime PYTHON:3.13
--sku F1 --logs
```

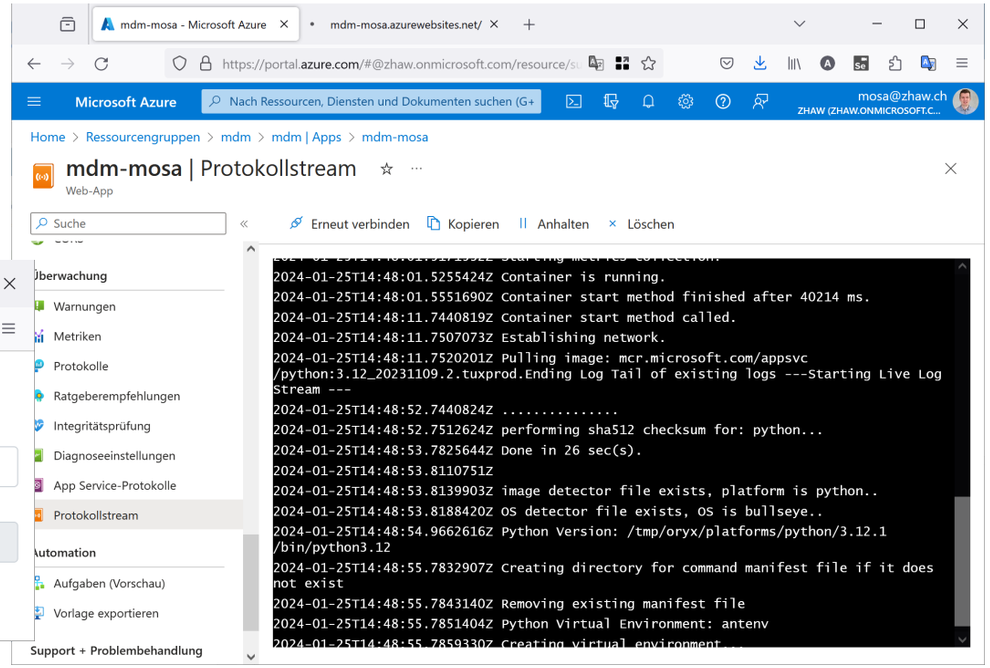
Hinweis: Vorher bestehende F1 Apps entfernen.

Logfiles (bei Problemen)

portal.azure.com

App-Service > Protokollstream





Azure Deployment Methoden

Azure App Service Deployment Methoden

- ZIP Deployment (az webapp up)
- Local Git Deployment
- App Service WebHook GitHub
- GitHub Actions
- Docker Image oder Docker Compose
- …